

AI-Based Visual Regression Testing for Web Applications

Detecting and classifying unintended UI changes across browsers, devices and locales

Tan Kia Yen · 0207077 · Final Year Project, Bachelor of Computer Science · Supervisor: Dr. Nor Rahayu Binti Ngatirin · UOW Malaysia

1 Introduction & Aims

A visual regression is a change to a web page that nobody meant to make: a button that vanishes after a CSS refactor, text that overflows only in Malay, a chart that renders blank on a phone. The functional tests still pass — the page is broken anyway.

Visual Regression Workbench captures a page, compares it against an approved reference image, and classifies what changed, so a reviewer only opens the changes worth opening.

Aims

- Detect unintended UI changes across browsers, devices and locales, automatically.
- Name the kind of change instead of only flagging pixels, so triage is not guesswork.
- Ship it as free, self-hosted software any team, student or NGO can run on a laptop.

2 Problem Statement

- Manual checking does not scale. 3 pages × 3 locales × 3 devices is 27 screens to inspect by eye every release. It gets skipped.
- Pixel-only tools cry wolf. Anti-aliasing, font hinting and animation produce diffs that mean nothing. Reviewers stop looking, then switch the check off.
- Commercial platforms are out of reach. Percy, AppliTools and Chromatic charge per snapshot and upload screenshots to a third-party cloud — a blocker for student teams, small companies, and anyone with data-residency rules.
- The layouts that break most are tested least. Non-English text and small viewports are where truncation, overlap and clipping appear first.

3 Target Users

- QA engineers and frontend developers in small and medium teams.
- Release owners who approve or block a deployment.
- University project teams with no QA budget and no licence.
- Public-sector and NGO web teams serving multilingual audiences who cannot send screenshots off-site.

4 App Features

- Dashboard for runs, baselines, reports and approvals, with live updates.
- Baseline versioning — old versions archived, rollback in one click.
- Locale, timezone and device-aware capture (en-US, ms-MY, zh-CN; desktop and phone).
- A DOM sidecar rides with every screenshot, giving structural comparison rather than pixel arithmetic.
- AI change classification: missing element, layout shift, text issue, colour regression, broken image.
- Playwright SDK drop-in — one line: `visualSnapshot(page, "homepage")`.
- Role-based access control: admin / developer / viewer.
- HTML report, JSON, JUnit output and GitHub commit-status checks.
- SQLite by default or PostgreSQL; deployed with docker-compose up.

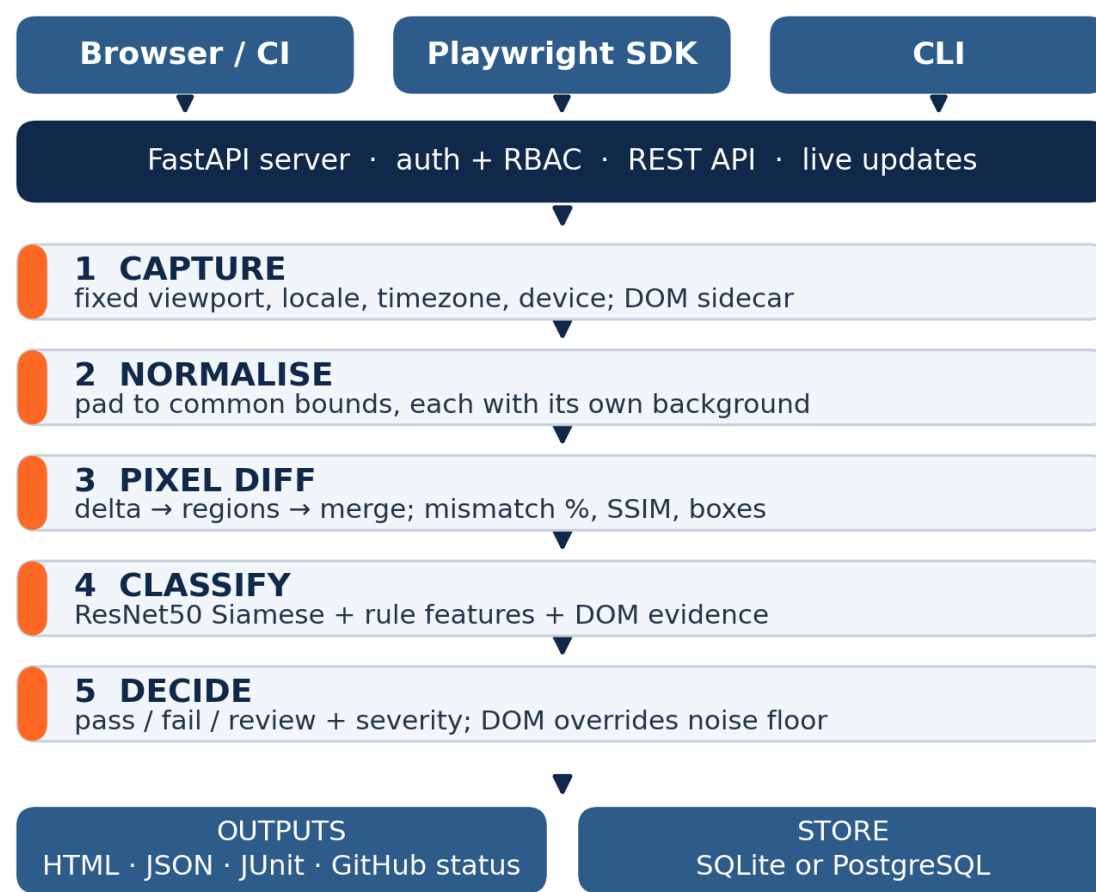
5 Methodology

Python + FastAPI, Playwright capture, PyTorch (ResNet50 Siamese) with OpenCV/SSIM, a React dashboard, SQLite or PostgreSQL. Every component is open source.

Evaluation — measured, not asserted

- Detection: baselines captured from the clean demo pages, then every page reloaded with `?defect=<mode>` — 9 defect types × 9 cases = 81 defective plus 9 clean controls. Ground truth comes from the injected mode, never from the model.
- Classification: 500 trials over 10 random seeds on real third-party pages with injected DOM mutations, with a 95% confidence interval.
- Every number here is re-checked by CI on each push; the build fails on any miss or false alarm.

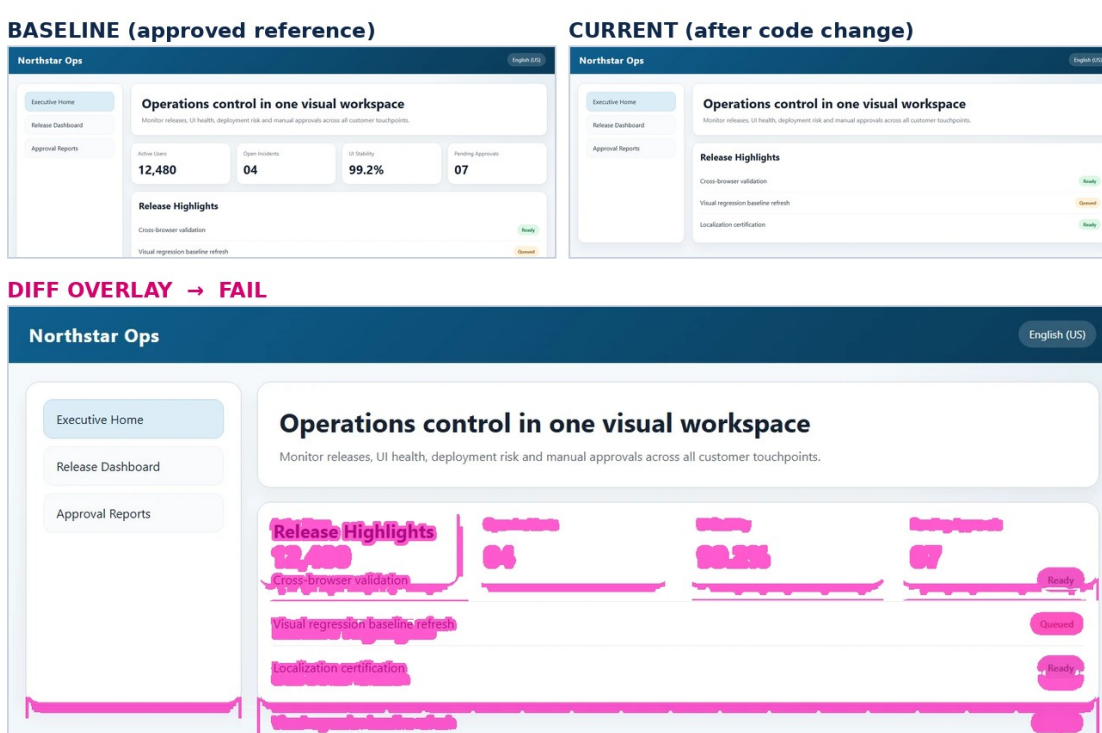
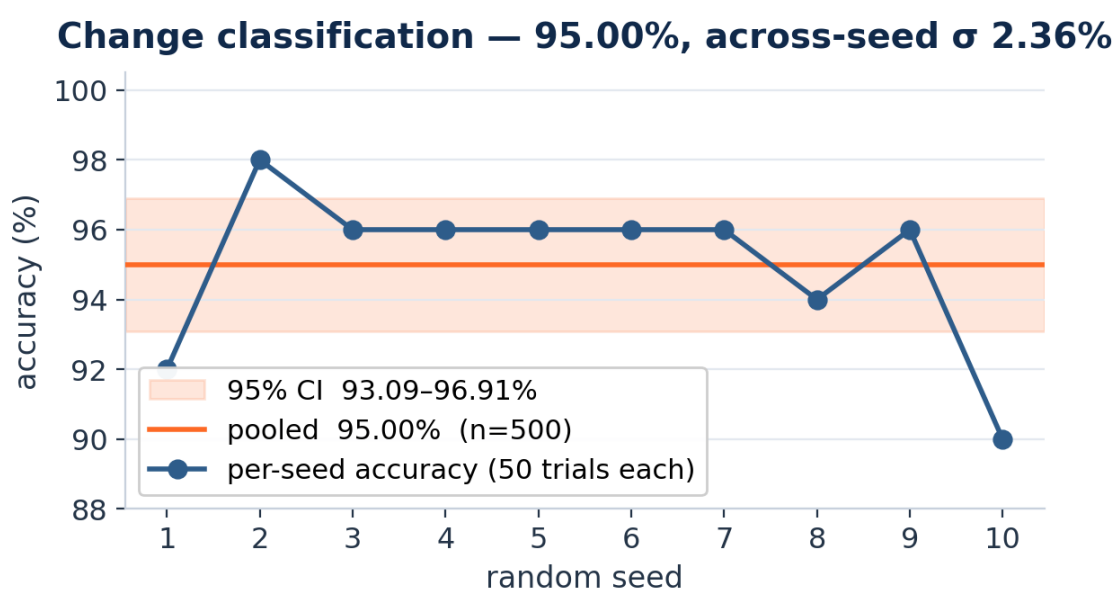
6 System Architecture & Workflow



7 Findings & Results



100% recall on all nine injected defect types — missing CTA, shifted card, theme shift, text truncation, overlay obstruction, broken image, misaligned fields, unreadable text, z-index issue.



Above: the metrics row was removed. Pixel diff 6.14% over one 1392 × 343 px region; labelled missing-element at 0.97 confidence; severity high; run fails.

Honestly reported: 15 of the 25 residual errors fall between label pairs describing one event from two angles — a removal reflows what sits below it, an overflow is also a layout change. Treating those as interchangeable scores 98.00%; the strict 95.00% stays the headline.

8 SDG Alignment



SDG 9 — Industry, Innovation and Infrastructure (primary)

Web interfaces are how people now reach banking, healthcare, enrolment and government services; a broken interface is an outage for the person who cannot finish the task. Targets 9.1 and 9.5 — this puts measured, automated release quality within reach of teams that cannot buy it.



SDG 4 — Quality Education

Free, self-hosted, and documented with decision records that explain why each choice was made. Target 4.4 — students practise industry release engineering on real tooling instead of reading about it.



SDG 10 — Reduced Inequalities

Capture is locale, timezone and device aware, and was evaluated in English, Malay and Chinese on desktop and phone — so layouts used by non-English speakers and low-end devices get the same scrutiny as the English desktop view.

9 Impact, Benefit & Feasibility

- Review effort collapses. Across 90 benchmark screens the reviewer opens only what is flagged, and 0 false alarms means no wasted triage.
- It catches what functional tests miss — all nine defect types, including ones that raise no console error and break no assertion.
- Zero licence cost, and no data leaves the organisation: screenshots stay on the team's own machine or server.
- Realistic to run. CPU-only inference, no GPU; SQLite on a laptop, PostgreSQL when a team needs it; docker-compose up is the whole deployment.
- It degrades safely. With no model present it falls back to pixel comparison and keeps detecting, so adoption never waits on training a model.
- Replicable anywhere — every component is open source, so any institution can stand up its own instance at no cost.

10 Conclusion & Recommendations

Detection is effectively solved on this benchmark — 81 of 81 defects caught, no false alarms — and classification stands at 95.00% with a stated confidence interval. The two are reported separately on purpose: they fail for different reasons, and one combined figure would hide which half is weak.

Recommendations & next work

- Close the 10 residual errors outside the overlapping label pairs and refine the change taxonomy.
- Extend the benchmark from the demo portal to production sites at scale.
- Add accessibility checks — contrast ratio, focus order — alongside the visual diff.
- Offer a shared community instance so student teams get visual QA with no setup at all.